Computer Programming (1)

Lecture No.8: Arrays

Lecturer: Dr. Khaled Mohammed bin Abdl
Department of Information Technology,
Faculty of Applied Science,
Seyion University

Semester 1, 2018

Content

- One-dimensional arrays
- Array elements
- Approaches of initialization arrays
- Out-of-Bound error
- Array of characters
- Passing arrays to functions
- Searching arrays
- Multidimensional arrays

Array

- Data structure containing related data items of the same type
  - Hold in consecutive group of memory locations
- "static" entity remain the same size throughout program execution
- Syntax: `dataType arrayName[arraySize];`
  - `int c[5];`

Array size (Number of elements)

Name of array (any valid identifier)

Type of array elements

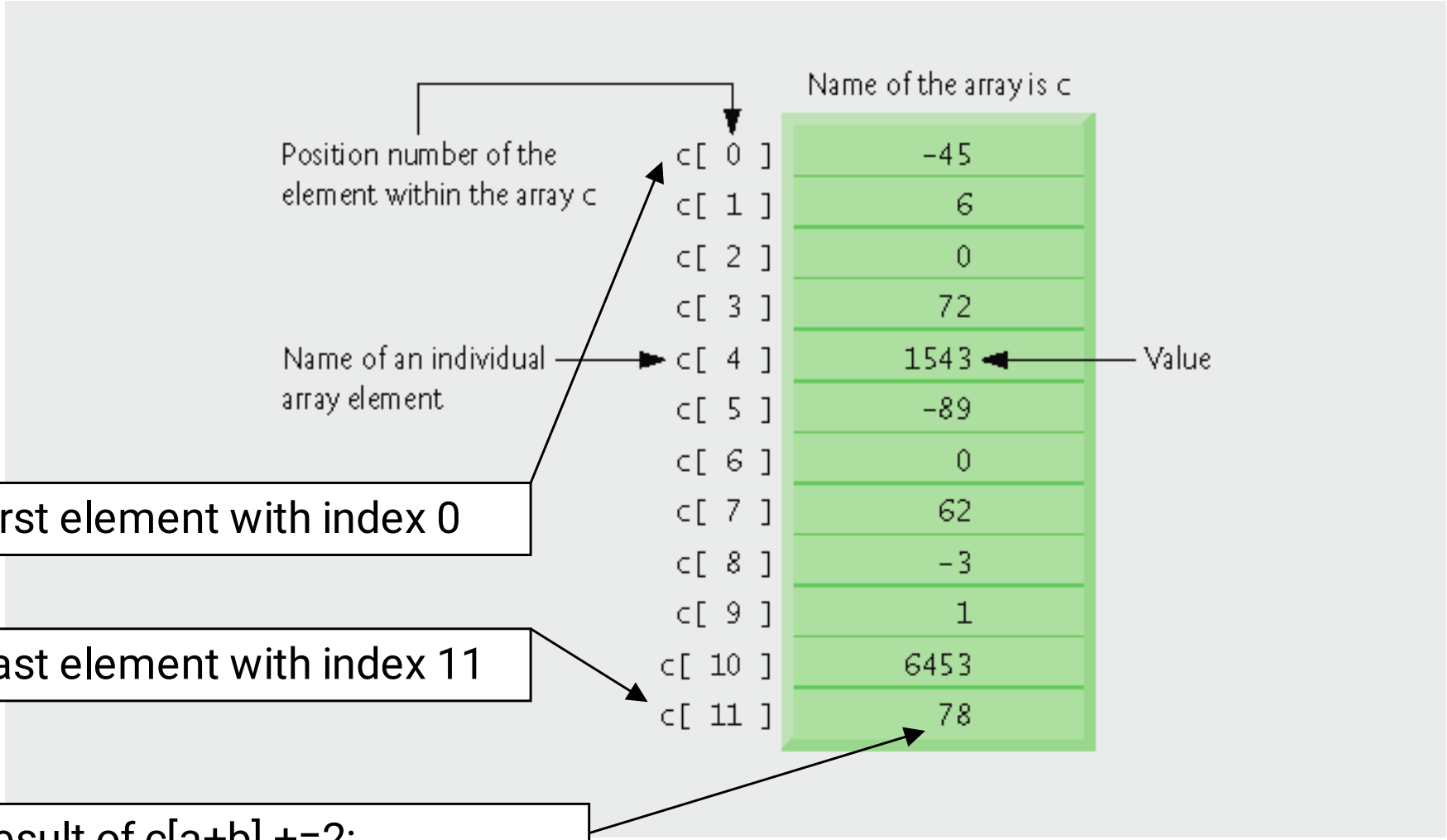
c[0]	?
c[1]	?
c[2]	?
c[3]	?
c[4]	?

Array Elements

- Data items in the array
- Each element has a position in the array called index or subscript
- First element has index 0 and last element has index arraySize-1
- To access any element, giving array name followed by index in [ ]
  - **c[0], c[1],** …etc
  - [ ] is an operator has same precedence and associativity of ( )
- Position number must be
  - Positive integer <u>OR</u> any expression that results in a positive integer

```
int a=5, b=6;
c[a+b] +=2; // Adds 2 to array element c[ 11 ]
```

Array Elements (cont.)



First element with index 0

Last element with index 11

Result of c[a+b] +=2;

will store 80 in c[11]

Array Initialization – Approach (1)

- Using a loop to initialize array's elements

```
// Fig. 7.3: fig07_03.cpp
// Initializing an array.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   int n[ 10 ]; // n is an array of 10 integers

   // initialize elements of array n to 0
   for ( int i = 0; i < 10; i++ )
      n[ i ] = 0; // set element at location i to 0

   cout << "Element" << setw( 13 ) << "Value" << endl;
```

Declare **n** as an array of **int**s with 10 elements

Each **int** initialized is to **0**

```
19
20     // output each array element's value
21     for ( int j = 0; j < 10; j++ )
22        cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;
23
24     return 0; // indicates successful termination
25 } // end main
```

```
Element          Value
      0              0
      1              0
      2              0
      3              0
      4              0
      5              0
      6              0
      7              0
      8              0
      9              0
```

n[j] returns **int** associated with index **j** in array **n**

Each **int** has been initialized to **0**

```cpp
// Fig. 7.4: fig07_04.cpp
// Initializing an array in a declaration.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // use initializer list to initialize array n
   int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };

   cout << "Element" << setw( 13 ) << "Value" << endl;

   // output each array element's value
   for ( int i = 0; i < 10; i++ )
      cout << setw( 7 ) << i << setw( 13 ) << n[ i ] << endl;

   return 0; // indicates successful termination
} // end main
```

```
Element         Value
      0            32
      1            27
      2            64
      3            18
      4            95
      5            14
      6            90
      7            70
      8            60
      9            37
```

Array Initialization – Approach(2)

- In declaration by using initializer list
  - Initializer list
    - Items enclosed in braces {}
    - Items in list separated by commas
    - Items themselves called initializers

```
int c[5] = { 10, 20, 30, 40, 50 };
```

c[0]	10
c[1]	20
c[2]	30
c[3]	40
c[4]	50

- If initializers are <u>more</u> than array elements (size)
  - Produce a compilation error

```
int c[5] = { 10, 20, 30, 40, 50, 60, 70 };
```

Array Initialization – Approach(2) (cont.)

- If initializers are **fewer** than array elements (size)
  - Remaining elements are initialized to zero

```
int c[5] = { 11 };
```

c[0]	11
c[1]	0
c[2]	0
c[3]	0
c[4]	0

- To initialize all array elements to 0
  - Explicitly initializes first element to zero
  - Implicitly initializes remaining four elements to zero

```
int x[5] = { 0 };
```

x[0]	0
x[1]	0
x[2]	0
x[3]	0
x[4]	0

  - Static array is initialized to zero by compiler
    - No need to initialized explicitly

Array Initialization – Approach(2) (cont.)

- Omitting array size in declaration, compiler determines array size based on number of items in initializer list

```
int c[] = { 10, 20, 30, 40, 50 };
```

  - How many elements in array c ? <u>OR</u> Array size?
  - Index values?
  - Initialized values?

Exercises :

Write C++ statements to accomplish each of the following:

a) Display the value of the seventh element of array **f**.

b) Input a value into element 4 of single-subscripted array **b**.

Array size As Constant variable


```
// Fig. 7.5: fig07_05.cpp
// Set array s to the even integers from 2 to 20.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // constant variable can be used to specify array size
   const int arraySize = 10;

   int s[ arraySize ]; // array s has 10 elements

   for ( int i = 0; i < arraySize; i++ ) // set the values
      s[ i ] = 2 + 2 * i;

   cout << "Element" << setw( 13 ) << "Value" << endl;

   // output contents of array s in tabular format
   for ( int j = 0; j < arraySize; j++ )
      cout << setw( 7 ) << j << setw( 13 ) << s[ j ] << endl;

   return 0; // indicates successful termination
} // end main
```

Declare constant variable **arraySize** using the **const** keyword

Declare array that contains 10 ints

Use array index to assign element's value

Element	Value
0	2
1	4
2	6
3	8
4	10
5	12
6	14
7	16
8	18
9	20

Example: Summation of the array elements


```
// Fig. 7.8: fig07_08.cpp
// Compute the sum of the elements of the array.
#include <iostream>
using std::cout;
using std::endl;

int main()
{
   const int arraySize = 10; // constant variable indicating size of array
   int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
   int total = 0;

   // sum contents of array a
   for ( int i = 0; i < arraySize; i++ )
      total += a[ i ];

   cout << "Total of array elements: " << total << endl;

   return 0; // indicates successful termination
} // end main
```

Declare array with initializer list

Sum all array values

```
Total of array elements: 849
```

```
The value of constant variable x is: 7
```

Out-of-Bound Error

- C++ has no array bounds checking
  - Does not prevent computer from referring to an element that does not exist
- Logic error
  - Referring to an element outside the array bounds

```
int main()
{
    const int arraySize = 10; // constant variable indicating size of array
    int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
    int total = 0;

    // sum contents of array a
    for ( int i = 0; i < arraySize; i++ )
        total += a[ i
                    i <= arraySize

    cout << "Total of array elements: " << total << endl;
```

Array of Characters (String)

- String is actually array of characters
- Using character arrays to store and manipulate strings
- Can be initialized using a string literal

```
char str[] = "first";
```

  - Array size is length of string **plus** a special string-termination character called the null character ('\0')

- Also can be initialized with individual character constants (with single quote) in an initializer list

```
char str[] = {'f','i','r','s','t','\0'};
```

  - Not providing null character produce logic error

Array of Characters (String) (cont.)

- It is possible to input a string directly into a character array from the keyboard using cin

```
char string1[20];
cin >> str;
```

  - cin >> may read more characters than the array can store which leads to lose data (logic error)

- Characters in array (string) are output with cout<< until null-terminated character is encountered

```cpp
// Fig. 7.12: fig07_12.cpp
// Treating character arrays as strings.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int main()
{
   char string1[ 20 ]; // reserves 20 characters
   char string2[] = "string literal"; // reserves 15 characters

   // read string from user into array string1
   cout << "Enter the string \"hello there\": ";
   cin >> string1; // reads "hello" [space terminates input]

   // output strings
   cout << "string1 is: " << string1 << "\nstring2 is: " << string2;

   cout << "\nstring1 with spaces between characters is:\n";

```

Store **"string literal"** as an array of characters

Initializing an array of characters using **cin**

Output array using **cin**

```
22    // output characters until null character is reached
23    for ( int i = 0; string1[ i ] != '\0'; i++ )
24       cout << string1[ i ] << ' ';
25
26    cin >> string1; // reads "there"
27    cout << "\nstring1 is: " << string1 << endl;
28
29    return 0; // indicates successful termination
30 } // end main
```

Loop until the terminating null character is reached

Accessing specific characters in the array

```
Enter the string "hello there": hello there
string1 is: hello
string2 is: string literal
string1 with spaces between characters is:
h e l l o
string1 is: there
```

Find the error(s) in each of the following statement:

a) Assume that: **char f[5];**

cin >> f; // User types hello

Passing Arrays to Functions

- Passing array as argument to function
  - by using array name without brackets [ ]

```
int c[5];     modifyArray(c, 5);
```

  - To process elements within function
    - Array size passed as argument
    - OR Place array size as global variable

- By default, arrays are passed by Reference
  - Called function can modify elements' values in original array
  - Function call actually passes starting address of array
    - So function knows where array is located in memory

Passing Arrays to Functions (cont.)

- Function parameter list must specify array parameter by [ ]

```
void modifyArray(int c[], int arraySize){}
```

  - Equivalent prototype

```
void modifyArray(int [], int);
```

- Array parameter may include the size of the array
  - Compiler will ignore it, though
    - Compiler only cares about the address of the first element

```
void modifyArray(int c[11], int arraySize){}
```

```cpp
// Fig. 7.14: fig07_14.cpp
// Passing arrays and individual array elements to functions.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

void modifyArray( int [], int ); // appears strange
void modifyElement( int );

int main()
{
   const int arraySize = 5; // size of array a
   int a[ arraySize ] = { 0, 1, 2, 3, 4 }; // initialize array a

   cout << "Effects of passing entire array by reference:"
      << "\n\nThe values of the original array are:\n";

   // output original array elements
   for ( int i = 0; i < arraySize; i++ )
      cout << setw( 3 ) << a[ i ];

   cout << endl;

   // pass array a to modifyArray by reference
   modifyArray( a, arraySize );
   cout << "The values of the modified array are:\n";
```

Function takes an array as argument

Declare **5-int** array **array** with initializer list

Pass entire array to function modifyArray

```
30
31    // output modified array elements
32    for ( int j = 0; j < arraySize; j++ )
33       cout << setw( 3 ) << a[ j ];
34
35    cout << "\n\n\nEffects of passing array element by value:"
36       << "\n\na[3] before modifyElement: " << a[ 3 ] << endl;
37
38    modifyElement( a[ 3 ] ); // pass array element a[ 3 ] by value
39    cout << "a[3] after modifyElement: " << a[ 3 ] << endl;
40
41    return 0; // indicates successful termination
42 } // end main
43
44 // in function modifyArray, "b" points to the original array
45 void modifyArray( int b[], int sizeOfArray )
46 {
47    // multiply each array element by 2
48    for ( int k = 0; k < sizeOfArray; k++ )
49       b[ k ] *= 2;
50 } // end function modifyArray
```

Pass array element **a[3]** to function **modifyElement**

Function **modifyArray** manipulates the array directly

```
// in function modifyElement, "e" is a local copy of
// array element a[ 3 ] passed from main
void modifyElement( int e )
{
   // multiply parameter by 2
   cout << "Value of element in modifyElement: " << ( e *= 2 ) << endl;
} // end function modifyElement
```

Function **modifyElement** manipulates array element's copy

```
Effects of passing entire array by reference:

The values of the original array are:
  0  1  2  3  4
The values of the modified array are:
  0  2  4  6  8


Effects of passing array element by value:

a[3] before modifyElement: 6
Value of element in modifyElement: 12
a[3] after modifyElement: 6
```

Passing Arrays to Functions (cont.)

- Since passing array to function by reference
  - Called function can modify elements' values
  - To prevent modification of array in called function
    - Precede array parameter by const qualifier

```
Void modifyArray(const int [], int);
```

```cpp
// Fig. 7.15: fig07_15.cpp
// Demonstrating the const type qualifier.
#include <iostream>
using std::cout;
using std::endl;

void tryToModifyArray( const int [] ); // function prototype

int main()
{
   int a[] = { 10, 20, 30 };

   tryToModifyArray( a );
   cout << a[ 0 ] << ' ' << a[ 1 ] << ' ' << a[ 2 ] << '\n';

   return 0; // indicates successful termination
} // end main

// In function tryToModifyArray, "b" cannot be used
// to modify the original array "a" in main.
void tryToModifyArray( const int b[] )
{
   b[ 0 ] /= 2; // error
   b[ 1 ] /= 2; // error
   b[ 2 ] /= 2; // error
} // end function tryToModifyArray
```

Using **const** to prevent the function from modifying the array

Array **a** will be **const** when in the body of the function

Array cannot be modified; it is **const** within the body function

Searching Arrays

- Process of finding a particular element n(key value OR search key) in the array
- Linear search is one of searching techniques
  - Compares each element of array with search key
  - Work well with small arrays OR unsorted arrays
  - Inefficient for large arrays
- Binary search is another searching technique
  - Can be used with sorted arrays

```cpp
// Fig. 7.19: fig07_19.cpp
// Linear search of an array.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int linearSearch( const int [], int, int ); // prototype

int main()
{
   const int arraySize = 100; // size of array a
   int a[ arraySize ]; // create array a
   int searchKey; // value to locate in array a

   for ( int i = 0; i < arraySize; i++ )
      a[ i ] = 2 * i; // create some data

   cout << "Enter integer search key: ";
   cin >> searchKey;

   // attempt to locate searchKey in array a
   int element = linearSearch( a, searchKey, arraySize );

```

Function takes an array, a key value, and the size of the array as arguments

Function returns location of key value, -1 if not found

```
25    // display results
26    if ( element != -1 )
27       cout << "Found value in element " << element << endl;
28    else
29       cout << "Value not found" << endl;
30
31    return 0; // indicates successful termination
32 } // end main
33
34 // compare key to every element of array until location is
35 // found or until end of array is reached; return subscript of
36 // element if key or -1 if key not found
37 int linearSearch( const int array[], int key, int sizeOfArray )
38 {
39    for ( int j = 0; j < sizeOfArray; j++ )
40       if ( array[ j ] == key ) // if found,
41          return j; // return location of key
42
43    return -1; // key not found
44 } // end function linearSearch
```

Search through entire array

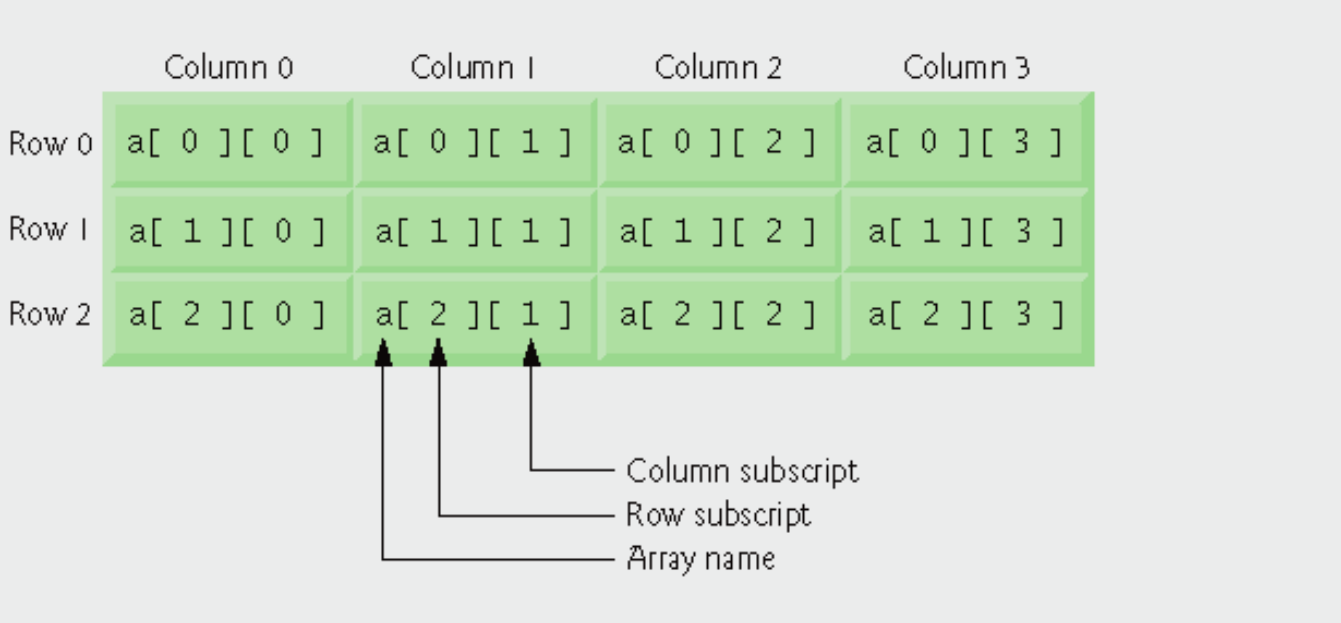
Return location if current value equals key value

```
Enter integer search key: 36
Found value in element 18
```

```
Enter integer search key: 37
Value not found
```

Multidimensional Array

- Multidimensional arrays with **two** dimensions
  - Called two dimensional or 2-D arrays
  - Represent tables of values with rows and columns
  - Elements referenced with two subscripts ([x][y])
  - Referencing a 2-D array element as c[x][y], incorrectly as c[x, y]



Initialization of Multidimensional Array

- Can be initialized in declaration much like one-dimension array
  - int b[2][2] = { { 1, 2 }, { 3, 4 } };
    - 1 and 2 initialize b[0][0] and b[0][1]
    - 3 and 4 initialize b[1][0] and b[1][1]
  - int b[2][2] = { { 1 }, { 3, 4 } };
    - Row 0 contains values 1 and 0 (implicitly initialized to zero)
    - Row 1 contains values 3 and 4
  - int b[2][2] = { 1, 2, 3, 4 };
    - 1 and 2 initialize b[0][0] and b[0][1]
    - 3 and 4 initialize b[1][0] and b[1][1]

Exercises :

a) Write a declaration for **t**.

b) How many rows does **t** have?

c) How many columns does **t** have?

d) How many elements does **t** have?

e) Write the names of all the elements in the second row of **t**.

f) Write the names of all the elements in the third column of **t**.

g) Write a single statement that sets the element of **t** in row 1 and column 2 to zero.

Multidimensional Array Parameters

- Size of first dimension is not required
  - As with a one-dimensional array
- Size of subsequent dimension is required
  - Compiler must know how many elements to skip to move to the second row

```
void printArray( const int a[][ 3 ] );
```

```cpp
// Fig. 7.22: fig07_22.cpp
// Initializing multidimensional arrays.
#include <iostream>
using std::cout;
using std::endl;

void printArray( const int [][ 3 ] ); // prototype

int main()
{
   int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
   int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
   int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };

   cout << "Values in array1 by row are:" << endl;
   printArray( array1 );

   cout << "\nValues in array2 by row are:" << endl;
   printArray( array2 );

   cout << "\nValues in array3 by row are:" << endl;
   printArray( array3 );
   return 0; // indicates successful termination
} // end main

// output array with two rows and three columns
void printArray( const int a[][ 3 ] )
{
   // loop through array's rows
   for ( int i = 0; i < 2; i++ )
   {
      // loop through columns of current row
      for ( int j = 0; j < 3; j++ )
         cout << a[ i ][ j ] << ' ';

      cout << endl; // start new line of output
   } // end outer for
} // end function printArray
```

```
Values in array1 by row are:
1 2 3
4 5 6

Values in array2 by row are:
1 2 3
4 5 0

Values in array3 by row are:
1 2 0
4 0 0
```

Common M-D Array Manipulation


```
for ( int col = 0; col < 4; col++ )
      a[ 2 ][ col ] = 0;
```

- What the previous code does?

```
total = 0;
for ( row = 0; row < 3; row++ )
      for ( col = 0; col < 4; col++ )
        total += a[ row ][ col ];
```

- What the previous code does?

End, Q & A